

```

        byte[] data = "This is the data to be
signed".getBytes();
        Signature signature = Signature.
getInstance("ML-DSA");
        signature.initSign(keyPair.getPrivate());
        signature.update(data);
        byte[] digitalSignature = signature.sign();
System.out.println("Digital Signature (ML-DSA): " +
Base64.getEncoder().encodeToString(digitalSignature));
        signature.initVerify(keyPair.getPublic());
        signature.update(data);
        boolean isVerified = signature.
verify(digitalSignature);
        System.out.println(" Verified Signature: " +
isVerified);
    } catch (NoSuchAlgorithmException |
InvalidKeyException | SignatureException e) {
        e.printStackTrace();
    }
}
}

```

## Flexible constructor bodies

Java 24 introduces flexible constructor bodies, providing developers with increased flexibility in defining constructors. This feature enhances control over object initialisation, thereby improving the readability and maintainability of code. It is part of the comprehensive enhancements in Java 24 designed to enhance the robustness of the language and improve its accessibility.

The advantages of utilising flexible constructor bodies encompass enhanced readability, secure initialisation, and adaptable validation. It is crucial to acknowledge that instance members (using `this`) or instance methods cannot be accessed prior to invoking `super()` or `this()`.

```

class Parentclass {
    int value;
    Parentclass (int value) {
        this.value = value;
    }
}
class Childclass extends Parentclass {
    int data;
    // Flexible constructor body
    Childclass(int value) {
        // Initialize fields and validation before calling super()
        if (value <= 0) {
            throw new IllegalArgumentException("Value must be
positive");
        }
        this.data = value * 2;
    }
}

```

```

// Explicit constructor invocation
super(value);
// Additional statements after super() are allowed
System.out.println("Value: " + value + ", Data Value:
" + data);
}

Childclass() {
    this(10);
    System.out.println("Secondary constructor called");
}
public static void main(String[] args) {
    Childclass obj1 = new Childclass(5); // Output: data :
5, Value: 10
    System.out.println("data: " + obj1.data + ", value: "
+ obj1.value);
    Childclass obj2 = new Childclass();
}

```

The `Childclass` constructor checks the value before calling the `Parentclass` constructor with `super(value)`. This validates the data and ensures the object is correctly initialised before the superclass constructor runs.

## JavaMoney API

The `JavaMoney` API in Java 24 provides a comprehensive framework for handling currencies and monetary amounts. This API simplifies the development of financial applications.

```

import javax.money.Monetary;
import javax.money.MonetaryAmount;
public class JavaMoneyExample {
    public static void main(String[] args) {
        MonetaryAmount amount = Monetary.getDefaultAmountFactory().
setCurrency("USD").setNumber(100).create();
        System.out.println("Monetary Amount: " + amount);
    }
}

```

Java 24 also introduces several other features, such as enhanced pattern matching, new concurrency utilities, and improvements to the Java standard libraries. These additions make Java 24 a comprehensive and robust platform for modern application development.

One of the standout features of Java 24 is its continued emphasis on performance optimisation. The introduction of new garbage collection algorithms and improvements in the JVM's execution engine will enhance application performance, particularly for large-scale, high-throughput systems.

■■ Continued to page...50